MANCHESTER 1824
The University of Manchester

Applications of LLMs

Jingyuan Sun
Lecturer of Natural Language Processing
Department of Computer Science
The University of Manchester

MANCHESTER 1824
The University of Manchester

Agenda

- **Agenda:**
  - The Transition from Foundation Models to Applications
  - Core NLP and Text-Based Applications
  - Domain-Specific Transformations (Code, Healthcare, Law)
  - LLMs as Autonomous Agents (Tool Use and Reasoning)
  - Deployment Challenges and Ethical Considerations
- **Learning Outcomes:**
  - **Map** core LLM capabilities to specific real-world industry use cases.
  - **Evaluate** the appropriate adaptation strategy (Prompting vs. RAG vs. Fine-Tuning) for different applications.
  - **Analyze** the architecture of LLM agents and tool-augmented systems.
  - **Critique** current limitations regarding deployment, cost, and safety.

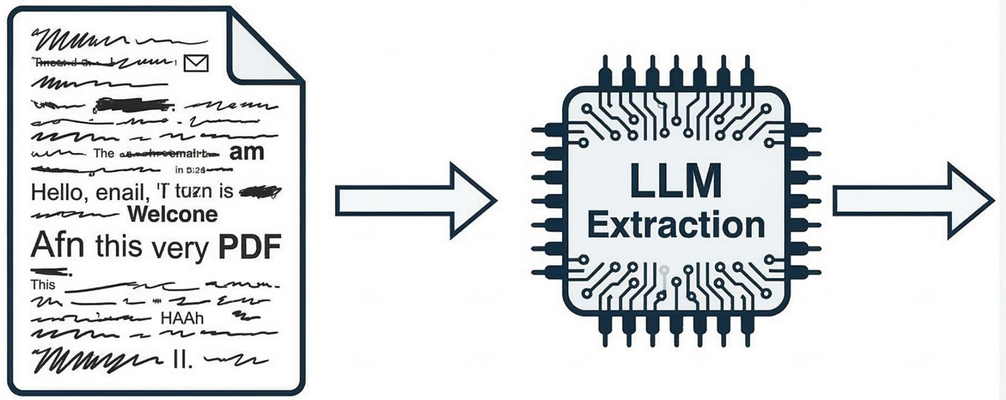
MANCHESTER 1824
The University of Manchester

Core Application 1: Advanced Text Summarization

- **The Evolution of Summarization:**
  - *Pre-LLM Era:* Extractive summarization (copying sentences) via smaller encoder models (like BERT).
  - *LLM Era:* Highly abstractive summarization, capable of synthesizing meaning.
- **Key Application Patterns:**
  - **Meeting Transcripts:** Converting raw speech-to-text logs into structured action items.
  - **Document Routing:** Summarizing customer support tickets to categorize and route to the correct human agent.
- **Evaluation Connection:**
  - How do we know the summary is good? As discussed in Week 6, we use metrics like ROUGE, but increasingly rely on **LLM-as-a-judge** (using strict evaluation prompts) to check for *factual consistency* and prevent hallucination.

Core Application 2: Structured Data Extraction

- **The Problem:** 80% of enterprise data is unstructured (emails, PDFs, reports).
- **The LLM Solution:** Using LLMs for Zero-Shot or Few-Shot Information Extraction (IE).
  - **Named Entity Recognition (NER):** Extracting companies, monetary values, and dates without training custom BERT models.
  - **Relation Extraction:** Identifying the relationship between entities (e.g., *Company A [ACQUIRED] Company B*).
- **Implementation Strategy:**
  - Use structural prompting (e.g., instructing the model to output strict JSON).
  - *Challenge:* Output parsing errors. Models may append conversational text ("Here is your JSON:") which breaks downstream code.




```
[
  "sender": "email@example.com",
  "subject": "Meeting Update",
  "date": "2023-10-27",
  "extracted_entities": {
    "entities": {
      "type": "Meeting"
    },
    "object_ennets": {
      "sendr": "fhread",
      "date": "Meeting"
    }
  }
}
```

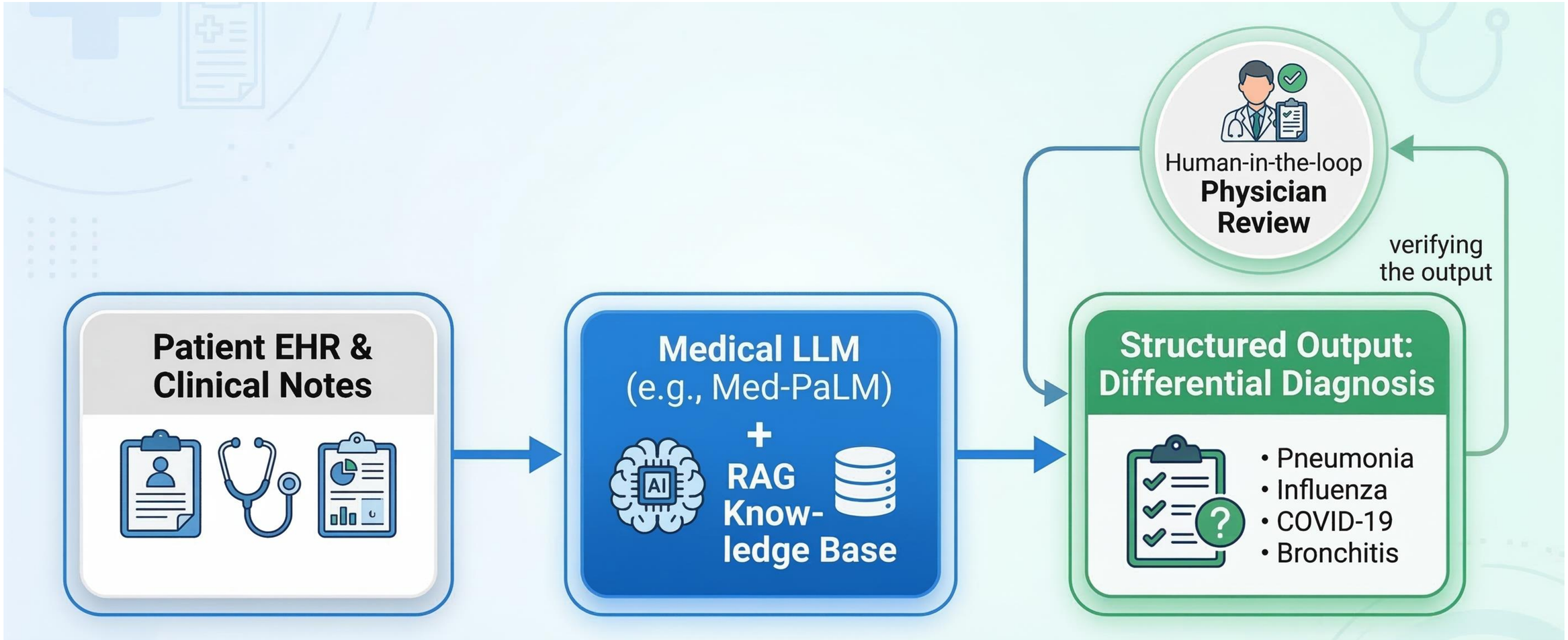
Mini-Quiz 1: Application Architecture

- **Scenario:** You are building an internal company tool to help HR instantly answer employee questions about the 200-page company benefits handbook.
- **Which architecture is the most appropriate and cost-effective?**
  - **A)** Pretrain a new foundational model from scratch on HR documents.
  - **B)** Use a general Instruction-Tuned LLM and Retrieval-Augmented Generation (RAG) with the handbook.
  - **C)** Use a standard Zero-Shot Prompt asking the model to guess the company policies.
  - **D)** Supervised Fine-Tune (SFT) an LLM solely on the HR handbook.

Domain-Specific Application: Software Engineering

- **Code Generation and Copilots:**
  - LLMs trained heavily on GitHub/StackOverflow (e.g., GitHub Copilot, CodeLlama).
  - *Tasks:* Autocompletion, writing unit tests, translating code between languages (e.g., Python to Rust).
- **Code Review and Bug Fixing:**
  - Using LLMs to detect security vulnerabilities or suggest performance optimizations.
- **Why LLMs excel at Code:**
  - Code has strict syntax and logic, reducing the ambiguity present in natural language.
  - *Evaluation:* Extremely objective. We don't need human graders; we evaluate generated code by simply running it through compiler/unit test suites (Pass@k metric).

Domain-Specific Application: Healthcare & Biomedicine



Class Discussion (5 Minutes)

- **Topic: The Generalist vs. Specialist Debate**
- Currently, state-of-the-art general models (like GPT-4) often outperform smaller, domain-specific fine-tuned models (like a 7B parameter legal-specific model) even on domain-specific tasks.
- **Discussion Question:** * Will the future of LLM applications be dominated by a few massive "God-models" accessed via API, or by thousands of smaller, locally hosted, highly specialized models?
  - *Hint: Consider data privacy, inference costs, and latency.*

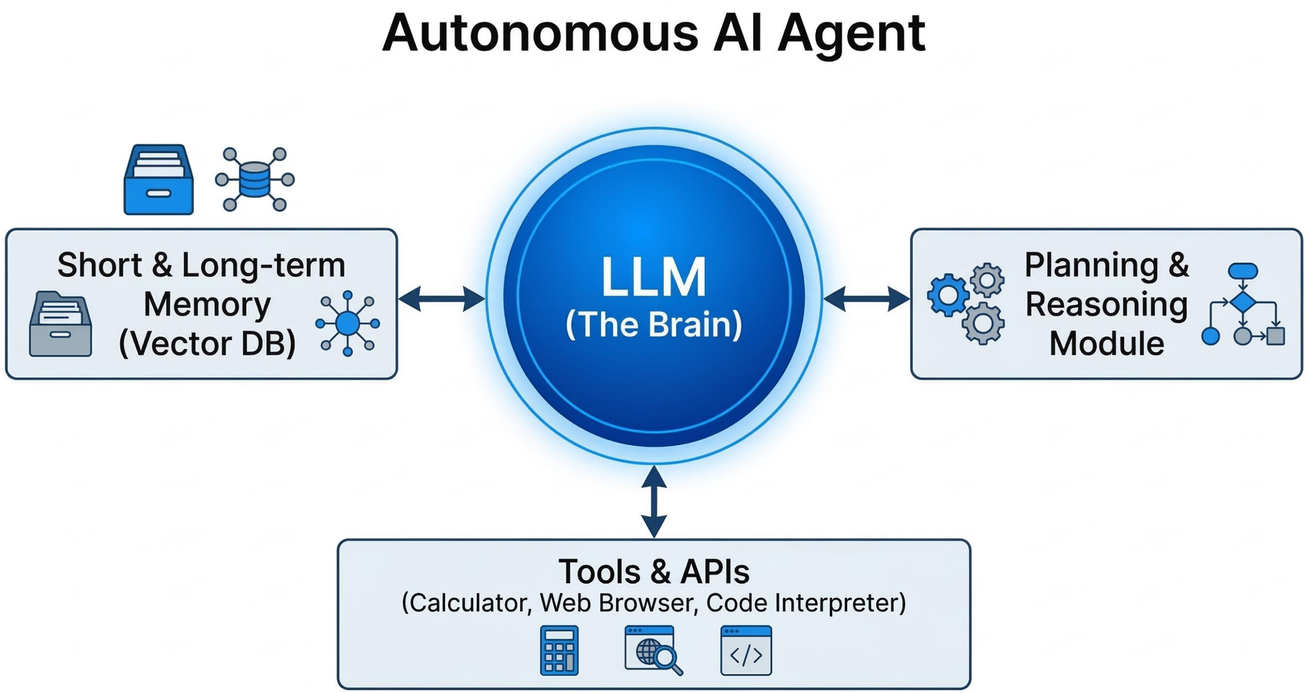
MANCHESTER 1824
The University of Manchester

Domain-Specific Application: Law and Finance

- **Legal Sector:**
  - *E-Discovery:* Scanning millions of documents for relevance to a subpoena.
  - *Contract Analysis:* Identifying risky clauses, non-standard terms, or missing indemnifications.
  - *Risk:* The "hallucinated legal cases" problem (models inventing fake precedent).
- **Finance Sector:**
  - *Algorithmic Trading:* Real-time sentiment analysis of financial news and SEC filings.
  - *Automated Reporting:* Synthesizing quarterly earnings calls into executive summaries.
  - *Adaptation technique:* Often heavily relies on RAG to ensure real-time data access (stock prices change by the second; parametric memory is useless here).

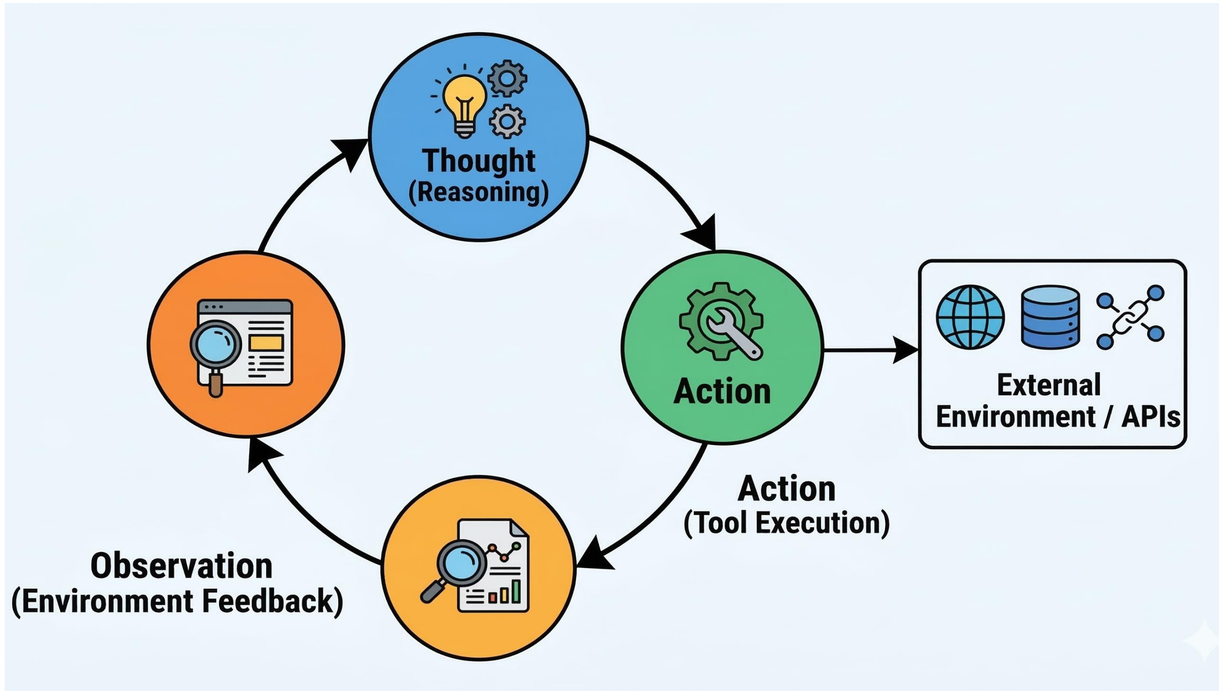
Application Paradigm Shift: LLMs as Agents

- **From Passive Responders to Active Agents:**
  - Standard LLM: Takes text, outputs text.
  - Agentic LLM: Takes a high-level goal, reasons about steps, interacts with external environments, and executes actions.
- **The Core Components of an LLM Agent:**
  - **1. Brain:** The LLM itself (handling reasoning and planning).
  - **2. Memory:** Short-term (context window) an Long-term (vector databases).
  - **3. Tools:** Functions the LLM can call (APIs, calculators, web browsers).



How Agents Work: ReAct Prompting

- **ReAct (Reasoning and Acting):**
  - A specific prompting paradigm that intertwines Chain-of-Thought reasoning with environmental actions.
- **The Loop:**
  - **Thought:** Model reasons about what to do next based on the user prompt.
  - **Action:** Model decides to call an external tool (e.g., SearchWikipedia("Eiffel Tower")).
  - **Observation:** The tool returns data to the model.
  - *(Loop repeats until the goal is achieved).*



- **Why this is powerful:** It overcomes the LLM's inability to do math or know current events by allowing it to use a calculator and a search engine.

MANCHESTER 1824
The University of Manchester

Multi-Agent Systems

- **The Concept:**
  - Instead of one massive prompt trying to do everything, instantiate multiple LLM agents with distinct personas and tools, and let them communicate.
- **Example System (Software Development):**
  - *Agent 1 (Product Manager):* Writes the system requirements.
  - *Agent 2 (Software Engineer):* Writes the Python code based on Agent 1's requirements.
  - *Agent 3 (QA Tester):* Reviews Agent 2's code, finds bugs, and sends it back for revision.
- **Benefits:**
  - Separation of concerns.
  - Self-reflection and self-correction through simulated debate.

MANCHESTER 1824
The University of Manchester

Mini-Quiz 2: Agents and Hallucinations

- **Question:** You ask an LLM Agent to calculate $345,678 \times 987,654$. Which of the following describes the safest "Agentic" approach?
- **A)** Prompt the LLM with "Let's think step-by-step" to calculate the math token-by-token.
- **B)** Provide the LLM with a Python interpreter tool, instruct it to write a Python script to multiply the numbers, execute it, and return the printed observation.
- **C)** Fine-tune the LLM on thousands of multiplication tables.

Deployment Bottlenecks: Why isn't LLM everywhere yet?

- Despite the amazing capabilities, real-world deployment faces severe engineering hurdles:
- **1. Inference Cost:**
  - Running a 70B parameter model costs significant GPU compute per token. Generative AI is structurally more expensive than traditional software.
- **2. Latency:**
  - Autoregressive generation (token-by-token) is slow. High latency degrades user experience in real-time applications (like voice assistants).
- **3. Reliability & Evaluation:**
  - Traditional software is deterministic (Input A always yields Output B).
  - LLMs are probabilistic. Continuous monitoring and robust evaluation pipelines (like the Inversion Prompting techniques covered in Week 6) are strictly required.

MANCHESTER 1824
The University of Manchester

Ethical and Safety Considerations in Applications

- **Bias and Fairness:**
  - Models amplify biases present in their training data (e.g., screening resumes and unfairly penalizing certain demographics).
- **Security (Prompt Injection):**
  - User-facing applications (like customer service bots) are highly vulnerable to malicious users altering the system prompt to leak data or output profanity.
- **Copyright and IP:**
  - Generative applications risk reproducing copyrighted code or text from their pretraining datasets.

MANCHESTER 1824
The University of Manchester

Discussion: The Future of Applications

- **Topic: Human-in-the-loop vs. Full Autonomy**
- Currently, most successful LLM applications act as "Copilots" (augmenting a human worker who reviews the final output).
- However, the industry is pushing toward "Agents" (full autonomy, executing workflows while humans sleep).
- **Discussion Prompt:**
  - In which industries is full LLM autonomy acceptable today?
  - Where must we permanently maintain a "human-in-the-loop" constraint?

MANCHESTER 1824
The University of Manchester

Summary & Q&A

- **Summary:**
  - LLM Applications extend base models using Prompting, RAG, and Tool-use to solve real business problems.
  - Core NLP tasks (Summarization, Extraction) are now largely solved by prompting.
  - Domain applications (Code, Medical, Legal) require strict adherence to facts, often leveraging RAG.
  - **Agents** represent the frontier: models that use tools, reason via ReAct, and interact with the environment.
  - Deployment is currently constrained by compute costs, latency, and safety evaluation.

MANCHESTER 1824
The University of Manchester

Multimodal LLMs

Jingyuan Sun
Lecturer of Natural Language Processing
Department of Computer Science
The University of Manchester

MANCHESTER 1824
The University of Manchester

Agenda & Learning Outcomes

- **Agenda:**
  - The Motivation for Multimodality
  - Core MLLM Architecture & The "Modality Gap"
  - Landmark Models (CLIP, Flamingo, BLIP-2, LLaVA)
  - Training Pipelines: Visual Instruction Tuning
  - Multimodal Prompting & Evaluation
- **Learning Outcomes:**
  - **Understand** the structural components of an MLLM (Encoder, Projector, LLM Backbone).
  - **Analyze** the two-stage training pipeline for modern vision-language models.
  - **Apply** multimodal prompting techniques (interleaved text/image).
  - **Evaluate** MLLMs using modern benchmarks and identify visual hallucination risks.

MANCHESTER 1824
The University of Manchester

What is a Multimodal LLM (MLLM)?

- **Definition:** * An MLLM is a language model capable of receiving, reasoning over, and (sometimes) generating multiple data modalities (text, images, audio, video).
- **The Goal:**
  - Retain the advanced reasoning and instruction-following capabilities of the LLM.
  - Grant the LLM "eyes" and "ears" to perceive the real world.
- **Focus of this Lecture:**
  - Vision-Language Models (VLMs), as they are currently the most researched and deployed subclass of MLLMs.

MANCHESTER 1824
The University of Manchester

Core Architecture of an MLLM

- Virtually all modern MLLMs share a tripartite (three-part) architecture:
- **1. The Vision Encoder:**
  - Extracts high-dimensional feature representations from raw image pixels.
  - *Example:* Vision Transformer (ViT).
- **2. The Connector (Projection Layer):**
  - Translates the visual features into the "language" (embedding space) of the LLM.
- **3. The LLM Backbone:**
  - The brain. Processes the translated visual tokens alongside standard text tokens to generate a text response.
  - *Example:* LLaMA, Vicuna.

MANCHESTER 1824
The University of Manchester

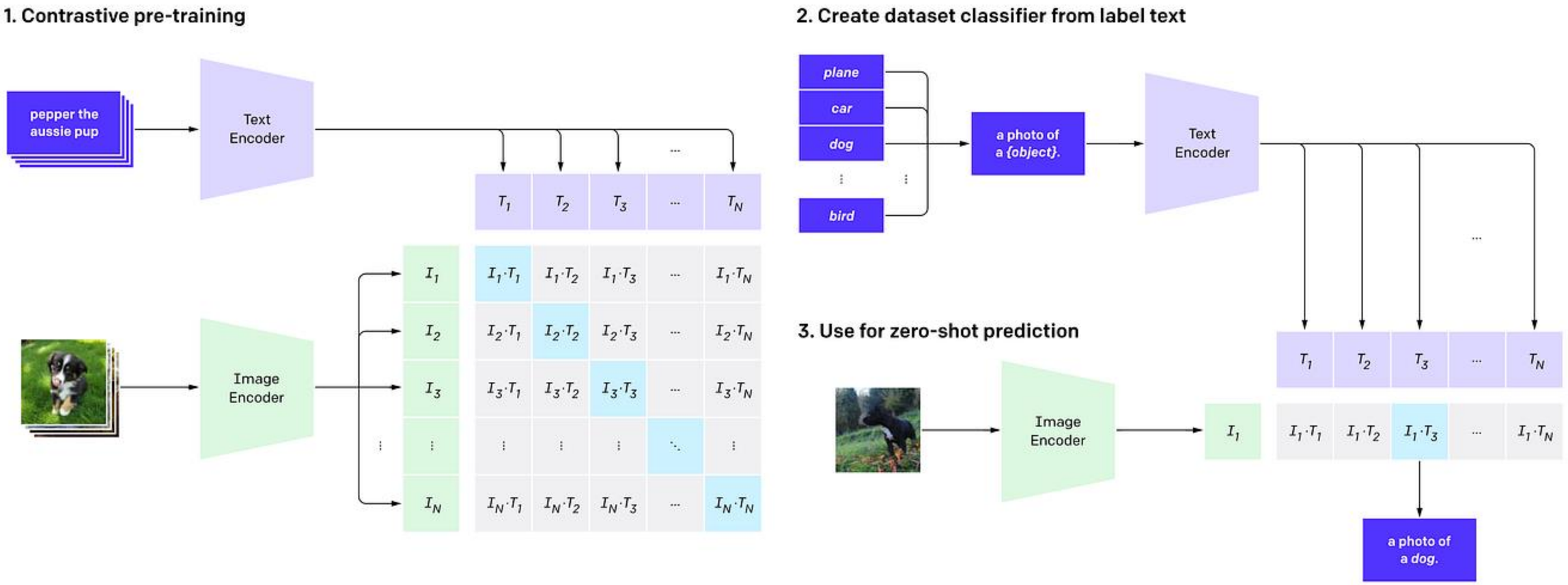
Component 1: The Vision Encoder (CLIP)

- **Contrastive Language-Image Pretraining (CLIP):**
  - A foundational model by OpenAI that maps images and text into the *same* vector space.
- **How it works:**
  - Trained on 400 million (Image, Caption) pairs.
  - Uses a Contrastive Loss function: maximizes the cosine similarity between the embedding of an image (e.g., a dog) and its correct text description ("a picture of a dog").
- **Why it matters for MLLMs:**
  - We freeze the CLIP Vision Encoder and use it as the "eyes" for our LLMs, because its visual features are already semantically aligned with human language.

Reference: Radford et al., "Learning Transferable Visual Models From Natural Language Supervision" (ICML 2021)

MANCHESTER 1824
The University of Manchester

CLIP Architecture Overview



CLIP pre-trains an image encoder and a text encoder to predict which images were paired with which texts in our dataset. We then use this behavior to turn CLIP into a zero-shot classifier. We convert all of a dataset's classes into captions such as "a photo of a dog" and predict the class of the caption CLIP estimates best pairs with a given image.

MANCHESTER 1824
The University of Manchester

Component 2: The Connector / Modality Gap

- **The Problem:**
  - The CLIP Vision Encoder outputs vectors of size D_{vision}.
  - The LLM expects word embeddings of size D_{text}.
  - These two spaces do not naturally communicate (The "Modality Gap").
- **The Solution:**
  - We introduce a lightweight, trainable neural network layer between them.
- **Connector Types:**
  - *Linear Projection:* A simple weight matrix (used in early LLaVA).
  - *MLP (Multi-Layer Perceptron):* A 2-layer network for better translation.
  - *Q-Former / Perceiver Resampler:* Compresses visual tokens to save context window space (used in BLIP-2, Flamingo).

MANCHESTER 1824
The University of Manchester

Mini-Quiz 1: Architecture Check

- **Question:** In a standard vision-language MLLM like LLaVA, which component is primarily responsible for performing complex logic, math, and generating the final conversational response?
- **A)** The Vision Transformer (ViT)
- **B)** The Contrastive Loss Function
- **C)** The LLM Backbone (e.g., Vicuna/LLaMA)
- **D)** The Projection Layer

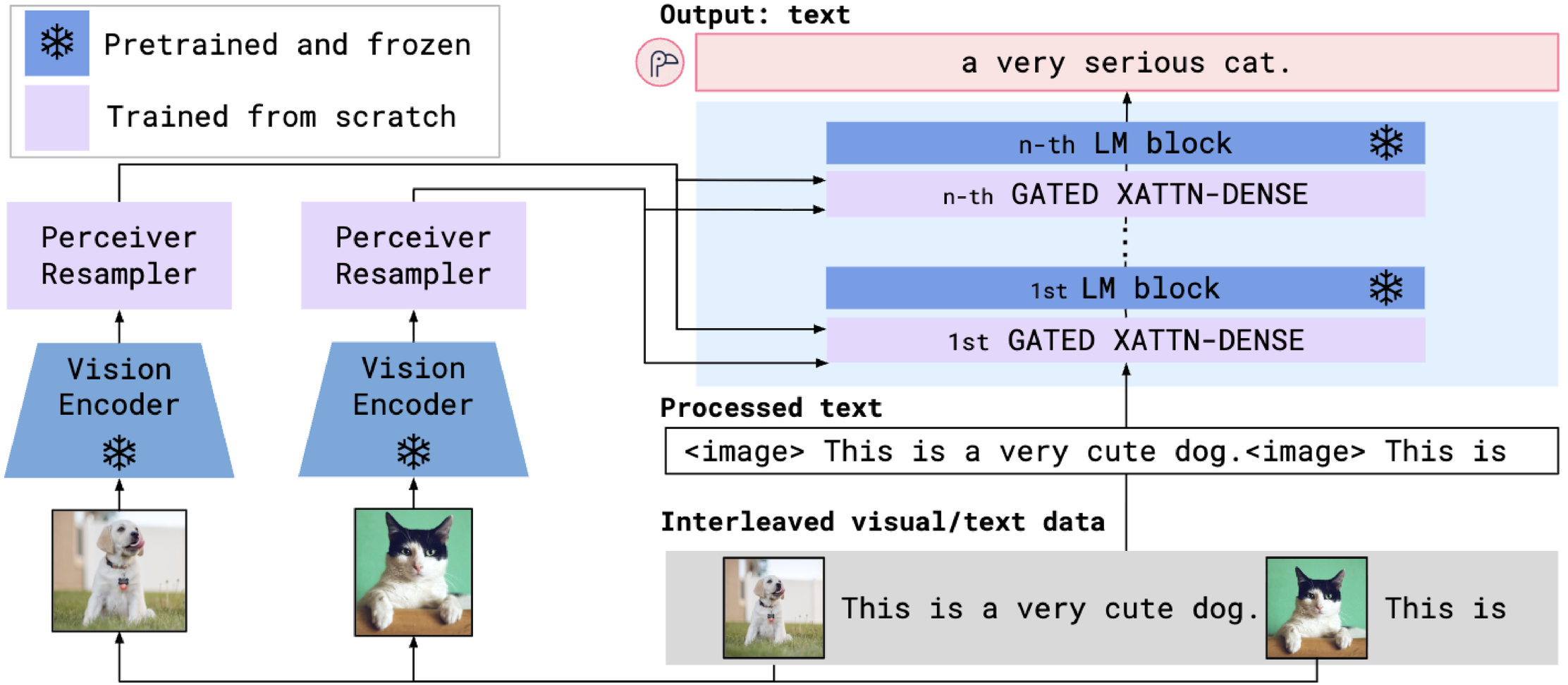
Landmark Model 1: Flamingo

- **The Breakthrough:** DeepMind's Flamingo was the first model to demonstrate highly effective interleaved image-text prompting.
- **Architecture Innovation:**
  - *Perceiver Resampler:* Compresses thousands of image tokens down to just 64 tokens, saving massive computational cost.
  - *Cross-Attention Layers:* Instead of feeding the image at the very beginning of the prompt, Flamingo injects visual information directly into the deep layers of the LLM.
- **Impact:** Proved that MLLMs could perform Few-Shot In-Context Learning (just like GPT-3), but with images.

Reference: Alayrac et al., "Flamingo: a Visual Language Model for Few-Shot Learning" (NeurIPS 2022)

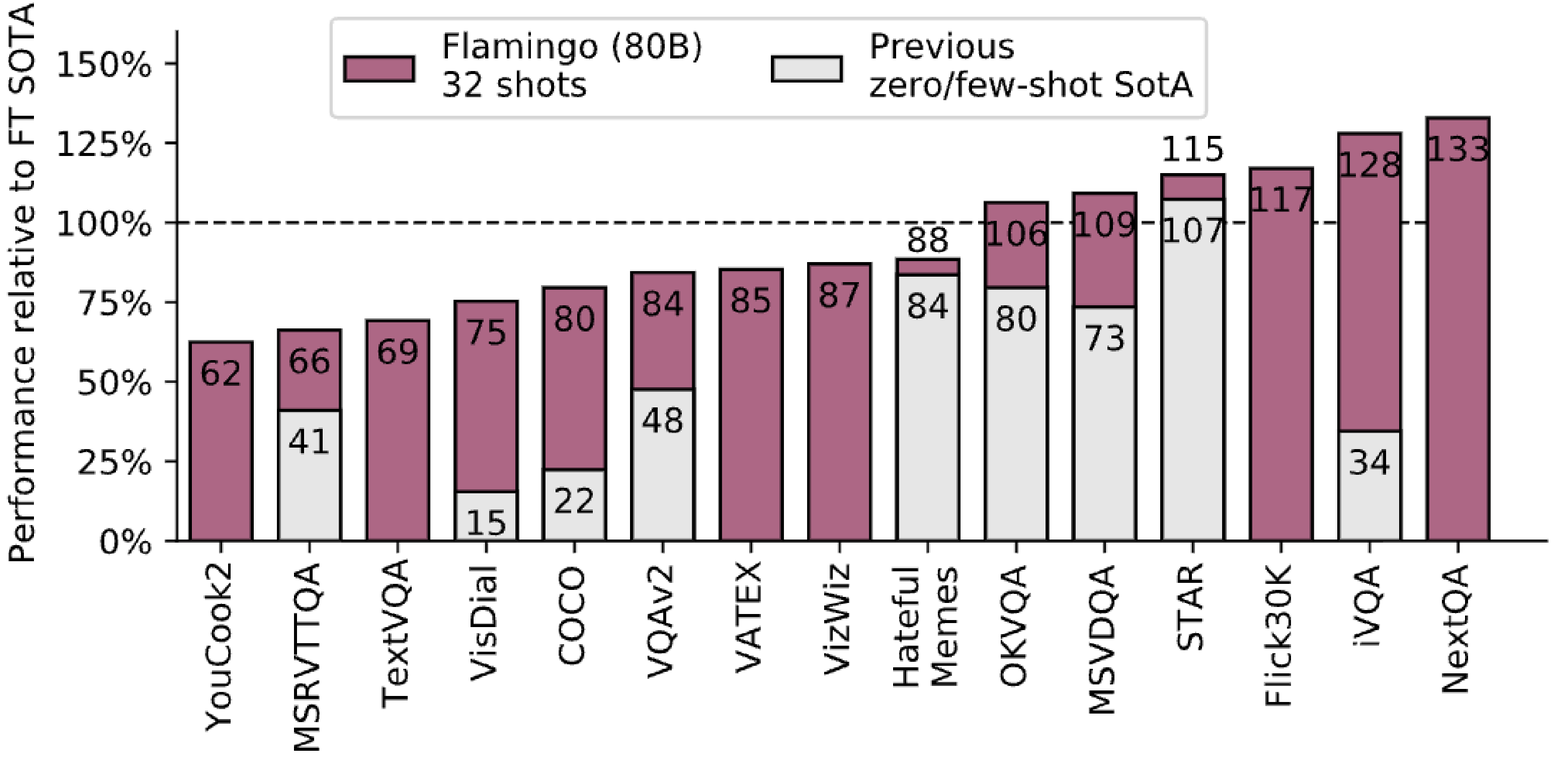
MANCHESTER 1824
The University of Manchester

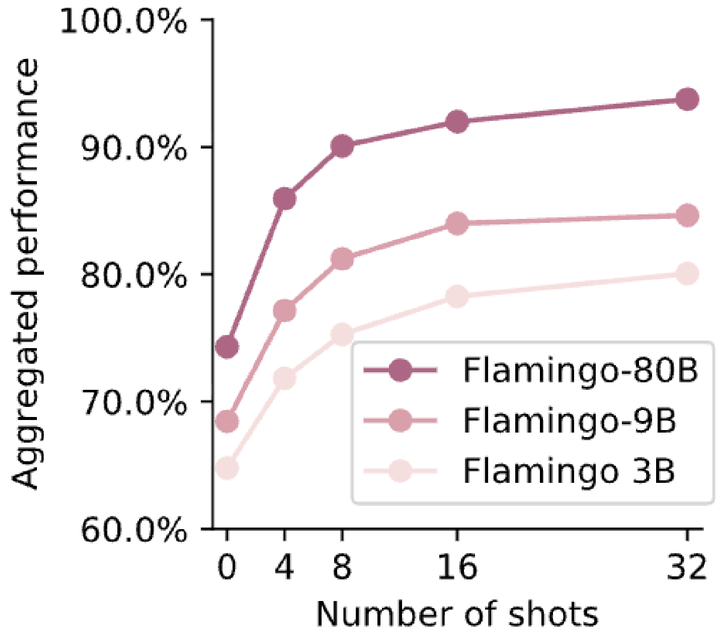
Flamingo Architecture Overview



MANCHESTER 1824
The University of Manchester

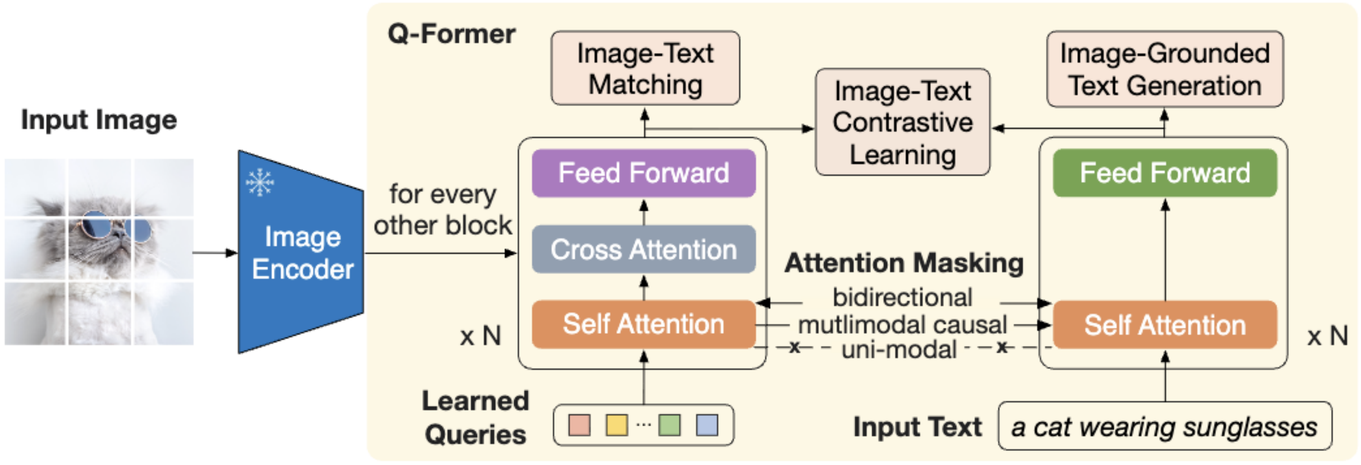
Flamingo Results Overview

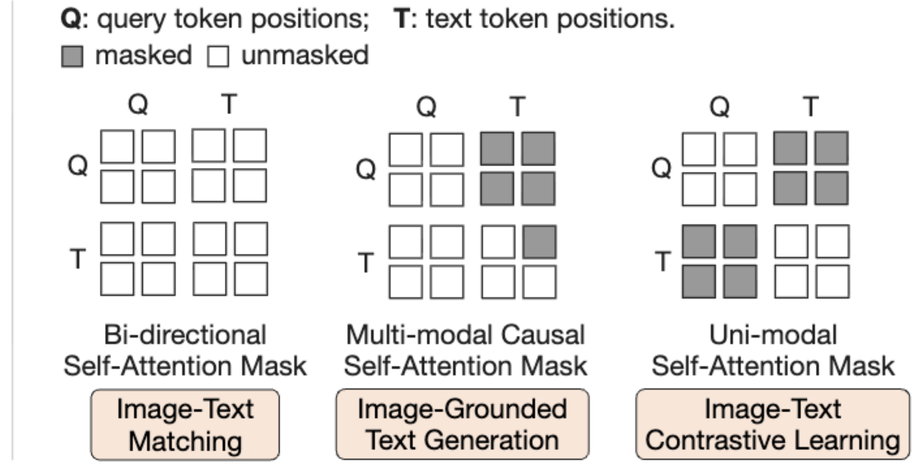


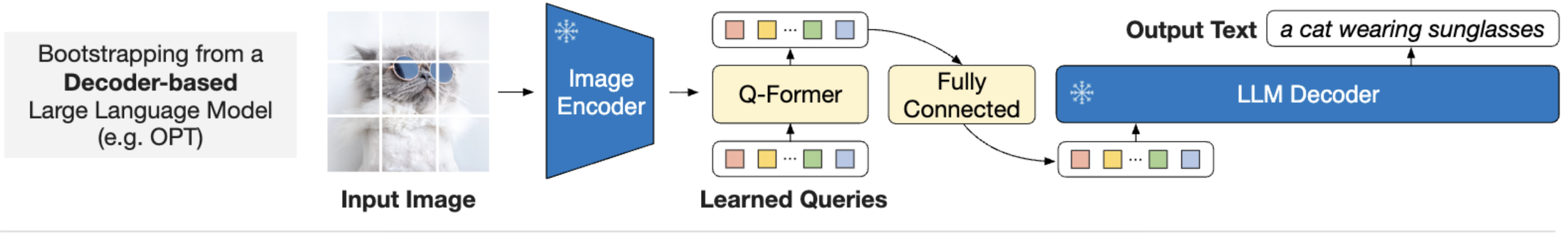


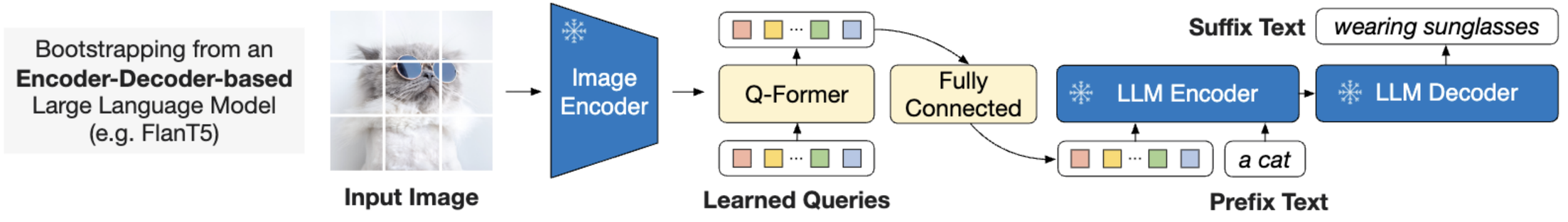
Landmark Model 2: BLIP-2

- *Reference: Li et al., "BLIP-2: Bootstrapping Language-Image Pre-training..." (ICML 2023)*
- **The Efficiency Breakthrough:**
  - Training massive LLMs from scratch is expensive.
  - BLIP-2 completely *freezes* both the Vision Encoder and the LLM Backbone.
- **The Q-Former (Querying Transformer):**
  - A lightweight, trainable module that learns to extract only the most relevant visual features required by the LLM.
- **Result:** Achieved state-of-the-art performance using significantly fewer trainable parameters, democratizing MLLM research for university labs.


Input Image
Image Encoder
Q-Former
Image-Text Matching
Image-Text Contrastive Learning
Image-Grounded Text Generation
for every other block
Feed Forward
Cross Attention
Self Attention
x N
Attention Masking
bidirectional
mutlimodal causal
uni-modal
Feed Forward
Self Attention
x N
Learned Queries
Input Text
a cat wearing sunglasses


Q: query token positions; T: text token positions.
masked unmasked
Q T
Q
T
Bi-directional Self-Attention Mask
Image-Text Matching
Multi-modal Causal Self-Attention Mask
Image-Grounded Text Generation
Uni-modal Self-Attention Mask
Image-Text Contrastive Learning


Bootstrapping from a **Decoder-based** Large Language Model (e.g. OPT)
Input Image
Image Encoder
Learned Queries
Q-Former
Fully Connected
LLM Decoder
Output Text
a cat wearing sunglasses


Bootstrapping from an **Encoder-Decoder-based** Large Language Model (e.g. FlanT5)
Input Image
Image Encoder
Learned Queries
Q-Former
Fully Connected
LLM Encoder
LLM Decoder
Suffix Text
wearing sunglasses
a cat
Prefix Text

MANCHESTER 1824
The University of Manchester

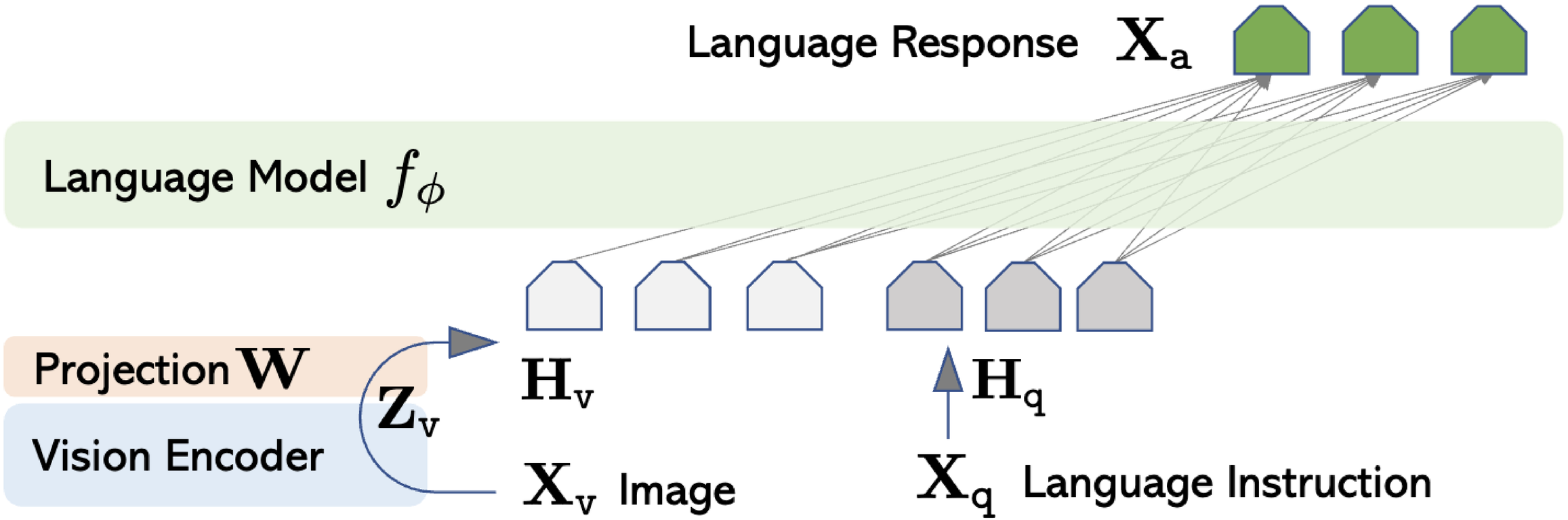
Landmark Model 3: LLaVA

- **Large Language-and-Vision Assistant (LLaVA):**
  - The open-source pioneer of Visual Instruction Tuning.
- **Simplicity is Key:**
  - Uses a frozen CLIP encoder.
  - Uses a simple Linear/MLP projection layer.
  - Uses a frozen Vicuna (LLaMA-based) LLM.
- **Why did it win?** The magic wasn't in the architecture; it was in the *data generation pipeline* (using GPT-4 to generate multimodal training data).

Reference: Liu et al., "Visual Instruction Tuning" (NeurIPS 2023)

MANCHESTER 1824
The University of Manchester

LLaVA Architecture Overview



MANCHESTER 1824
The University of Manchester

The MLLM Training Pipeline (Stage 1)

- Training an MLLM like LLaVA happens in two distinct stages.
- **Stage 1: Feature Alignment Pretraining**
  - **Goal:** Teach the LLM to understand the Vision Encoder's "language".
  - **Data:** Millions of simple (Image, Caption) pairs (e.g., CC3M dataset).
  - **Frozen:** Vision Encoder and LLM Backbone.
  - **Trainable:** *Only* the Projection Layer.
  - **Analogy:** Teaching an English speaker (LLM) to read a French dictionary (Vision features) by mapping words one-to-one.

The MLLM Training Pipeline (Stage 2)

- **Stage 2: Visual Instruction Tuning (Multimodal SFT)**
  - **Goal:** Teach the model to follow complex human instructions about images (e.g., "Why is this meme funny?").
  - **Data:** ~150k highly detailed, instruction-following multimodal conversations.
  - **Frozen:** Vision Encoder.
  - **Trainable:** Projection Layer *and* LLM Backbone.
- **Connecting to Week 6:** Just like standard Supervised Fine-Tuning (SFT) taught text LLMs to be assistants, Visual Instruction Tuning teaches MLLMs to be *visual* assistants.

MANCHESTER 1824
The University of Manchester

Class Discussion: Data Generation (5 mins)

- **Topic: How do we get Visual Instruction Data?**
- In 2023, the LLaVA authors needed 150,000 conversational Q&A pairs about images, but human labeling was too expensive.
- They used a text-only model (GPT-4) to generate data for their multimodal model.
- **Discussion Question:**
  - If GPT-4 (at the time) was text-only and couldn't see images, how did the authors use it to generate highly accurate, detailed visual Q&A data to train LLaVA?
  - *(Hint: Think about bounding boxes and captions).*

MANCHESTER 1824
The University of Manchester

Beyond Images: Audio and Video

- **Video as a Sequence of Images:**
  - Video MLLMs (like Video-LLaVA) treat video as a sequence of frames.
  - *Challenge:* Context window exhaustion. 30 frames of video = 30x the visual tokens.
- **Audio Encoders:**
  - Architectures like Whisper (Audio Transformer) can be projected into the LLM just like CLIP.
- **Any-to-Any Models:**
  - Emerging models (like GPT-4o or Gemini 1.5 Pro) are trained natively on multiple modalities from scratch, bypassing the strict "Encoder -> Projector" pipeline for more organic cross-modal understanding.

Evaluating MLLMs: Traditional Benchmarks

- How do we know if an MLLM is good?
- **1. VQAv2 (Visual Question Answering):**
  - Short, factual questions (e.g., "What color is the car?").
  - *Drawback:* Too simple for modern MLLMs.
- **2. OK-VQA (Outside Knowledge VQA):**
  - Requires world knowledge not present in the image.
  - *Example:* `` + "Which political party does this person belong to?"
- **3. Image Captioning (COCO):**
  - Generating descriptive text. Evaluated via n-gram overlap metrics like CIDEr or BLEU.

Evaluating MLLMs: Comprehensive Benchmarks

- Because modern MLLMs are conversational, old benchmarks fail to capture their abilities.
- **1. MME (Multimodal Evaluation):**
  - Tests perception (color, position, count) and cognition (commonsense, math, code).
  - Evaluates robustness using Yes/No prompts.
- **2. MM-Vet:**
  - Evaluates complex, multi-turn multimodal dialogue.
  - Uses **LLM-as-a-Judge** (usually GPT-4V) to grade the MLLM's responses based on accuracy, helpfulness, and reasoning.

Mini-Quiz 2: Evaluation Constraints

- **Scenario:** You evaluate your new MLLM on VQAv2.
- Ground Truth Answer: "Red"
- Your Model's Answer: "The car shown in the image is red."
- **Question:** What happens to your evaluation score under strict traditional string-matching metrics (like Exact Match)?
  - **A)** Score is 100% (Correct).
  - **B)** Score is 0% (Incorrect).
- **Answer: B.** * *Insight:* Instruction-tuned MLLMs are "chatty". Traditional benchmarks penalize them for full sentences. This is why LLM-as-a-judge is now the standard for MLLM evaluation.

MANCHESTER 1824
The University of Manchester

The Hallucination Problem in MLLMs

- **Visual Hallucination:** * When an MLLM confidently describes objects, attributes, or relationships that do *not* exist in the provided image.
- **Why does it happen?**
  - **Language Prior:** The LLM backbone is so powerful that it overrides the visual encoder. (e.g., It sees a dining table and hallucinates "chairs" because textually, chairs usually accompany tables).
  - **Connector Compression:** Small details get lost when reducing megapixel images to a few hundred tokens.

MANCHESTER 1824
The University of Manchester

Evaluating Hallucinations: POPE

- **Polling-based Object Probing Evaluation (POPE):**
  - A benchmark specifically designed to expose visual hallucinations.
- **Methodology:**
  - Frames evaluation as binary Yes/No questions.
  - *Adversarial Probing:* Asks about objects that frequently co-occur but are missing from the specific image (e.g., `` -> "Is there a mouse in the image?").
- **Finding:** Many early MLLMs failed POPE catastrophically, defaulting to "Yes" due to statistical priors.

Reference: Li et al., "Evaluating Object Hallucination in Large Vision-Language Models" (EMNLP 2023)

Class Discussion: The "Clever Hans" Effect

- *Context:* Clever Hans was a horse who appeared to do math, but was actually just reading the subtle body language of his trainer.
- **Discussion Question:**
  - Are MLLMs truly "seeing" the image, or are they experiencing a Clever Hans effect where the user's prompt gives away the answer?
  - *Example:* If I prompt: "What is the man holding in his left hand?", does the model know there is a man, or does it just assume there is based on my question? How can we design prompts to prevent this?